Checking or Recomputing? Two Cost Regimes in Verifying AI-Generated Formal Artifacts

Anonymous

Abstract

Checking a stored proof can avoid its original search, while checking a computational certificate can repeat the calculation that justifies it. We measure these costs in a pinned, published AI-generated Lean library and use the findings to design ProofServe, an original-kernel check and audit tool. Matched target windows retain 80.28–82.35% of source-compilation wall time for two computational certificates, versus 0.46–6.54% for two ordinary controls. Across 31 module-process pairs, the median checker/compiler CPU ratio is 0.451, with overlapping source-proxy classes. A completed 42-module probability sample estimates 279.12 CPU-hours of library compilation; its nominal 95% interval, 95.18–463.06 hours, remains wide. The default import graph omits 19.78% of source modules and 33.30% of lexical kernel-decision sites. ProofServe combines dependency-aware parallel replay, restartable segments and receipts recording scope, axioms and costs. Its contribution is a usable integration of established mechanisms and an explicit evaluation contract; release scaling and external checking results remain pending.

1 Introduction

Proof-embedded computation can preserve expensive work across the boundary between producing and checking an AI-generated proof. Lean validates elaborated terms rather than repeating tactic search, but definitional equality can still evaluate a program inside a term. Users therefore need both a cost model and a precise account of what a successful project check covers.

Existing work already establishes the value of independent checking. Lean 4.35 ships `lake check`, following the FRO's external-checking agenda [8, 7]. A public report on the same OpenAI library performs a complete cloud build and extensive independent checking [10]. It proposes kernel computation as an explanation for expensive checks from three source examples. Our empirical increment is a matched module-level measurement of that explanation, a probability sample with explicit uncertainty, and a coverage analysis at a different corpus revision. Our systems contribution is ProofServe's original-kernel workflow and auditable receipts, evaluated through its release-form interface. The four questions follow these boundaries:

RQ1 How do checking costs differ between stored mathematical proofs and computational certificates?

RQ2 What compilation cost and coverage can a fixed library sample establish?

RQ3 How does parallel original-kernel checking trade wall time, CPU work and additional trusted code?

RQ4 What do default builds, repeated verdicts and axiom receipts establish about verification coverage?

2 Method and workload

A fixed probability sample and matched checking windows separate population cost from selected-case behavior. Our primary corpus is the published OpenAI mathematics repository [11], pinned at `4eaee875`: 163,097 source modules, Lean 4.34.1 and Mathlib `d13f23b7`. Two original draws in each of 21 strata combine source-size percentile, import depth and an explicit kernel-decision marker. All 42 now have completed compilation costs. Prerequisites and selected hot examples retain their separate identities and weights.

Source cost includes a module's imports, tactics, kernel checking and artifact emission. Separate prerequisite preparation, C compilation and build-driver work are outside this clock. CPU means user plus system time. Whole-process replay and incremental target windows are distinct measurements; Figure 1 labels both. Timed work runs one task at a time on CPUs 0–7 of an i9-14900K, retaining file caches. Lean runs omit `-M`, including `-M0`; external memory protection is recorded separately. Earlier in-process memory checks and missing prototype persistence marking were our own method defects.

The public package `data/openai-math-4eaee875/` records sample identities, raw-cost provenance, clocks and estimates. This paper pins its original-sample summary to `1cc14067`. Historical replay measurements explain cost boundaries; system performance must come from committed release-tool results in `checker/results/` and `data/external-projects/`.

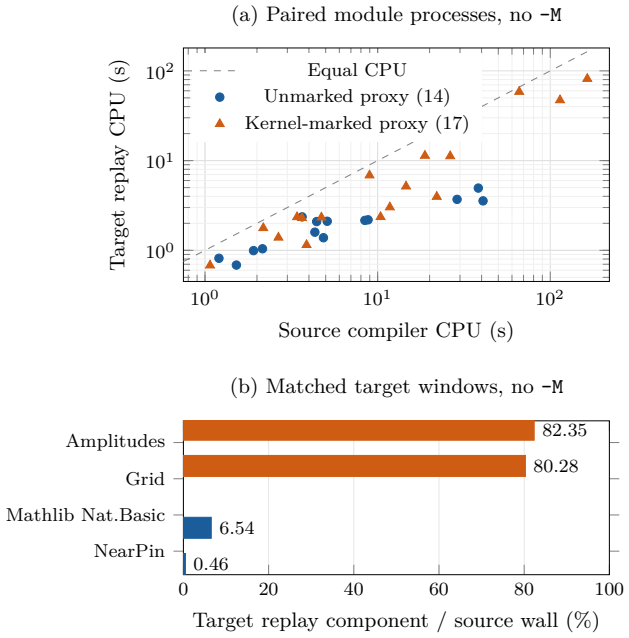


Figure 1: Two views of checking cost, all without `-M`. (a) Thirty-one complete module-process pairs; colors are source-marker proxies, whose distributions overlap. Imports are trusted in target replay. (b) Ordinary controls (blue) and computational certificates (orange), using matched target read/map/replay wall windows after dependency preparation. CPU in (a) and wall time in (b) have separate denominators. Design implication: expose the checking stage and amortize preparation; a closure average can hide an expensive certificate.

3 Two costs and their population weight

3.1 Computational certificates retain work

Matched target windows show expensive replay for computational certificates and cheap replay for ordinary controls. In Figure 1(b), Grid and Amplitudes retain 80.28% and 82.35% of their source compiler's wall time. Their replay components are 255.489 and 110.026 seconds; almost all of each is inside the kernel replay call. Mathlib Nat.Basic and NearPin retain 6.54% and 0.46%. Each target component reads, maps and checks that target after dependency preparation; each denominator is that module's compiler process. These shared boundaries quantify the expensive kernel replay anticipated by the report's computation hypothesis in the selected cases [10].

The mechanism is visible in the proof boundary. A tactic can search for a mathematical derivation and emit a term that records its successful steps. Replaying that term validates those steps without reconstructing the search. A finite certificate justified through `of_decide_eq_true` instead asks the kernel to validate the decision procedure's result through reduction. The compact wrapper can therefore retain evaluation over a large finite object. Our target measurement holds the declared input and original checking operation fixed and locates that retained cost in the replay window. Its practical consequence is that source length and a cheap closure average are weak planning signals for an individual certificate.

The broader paired panel shows variation rather than a clean binary split. Across 31 original modules, checker/compiler CPU has median 0.451: 0.391 for the 14 unmarked and 0.498 for the 17 marked modules. An explicit comment/string-masked `decide +kernel` site is a reproducible proxy for computation, while ordinary `decide` can also encode substantial evaluation [2, 5]. For example, the unmarked Ramsey K10 certificate uses 356.44 checker CPU-seconds against 1,725.13 source CPU-seconds (20.66%). A 20-module sample of the retained 190-module Ramsey prefix uses 35.91% in aggregate. That prefix covers 190 of 444 targets and is dependency-ordered; its sample describes the retained prefix.

Whole closures dilute expensive targets with cheaper dependencies. Checkpointed Falconer checking costs 2,038.44/32,292.60 CPU-seconds (6.31%); matched checkpointed Mathlib costs 91.15/1,018.22 (8.95%). Their separate wall ratios are 9.26% and 14.67%. SubexponentialSAT primary accounting reports 3,399.30 checker CPU-seconds for all 14,721 accepted modules, or 7.25% of complete source work. Using only the 3,977 new targets' source cost gives 15.79% for the same whole-closure checker numerator. Its independent reference controls remain pending. This denominator change illustrates why a single ratio cannot stand for every boundary.

3.2 A complete sample still gives a wide interval

Completing all 42 original draws yields a full-frame estimate of 279.12 compiler CPU-hours, with a nominal 95% interval of 95.18–463.06 hours. The stratified estimator sums each stratum's module count times its sample mean; its finite-population variance sums $N_h^2(1 - n_h/N_h)s_h^2/n_h$. A Student-$t$ approximation with Satterthwaite degrees of freedom 2.61 produces the interval. The marked strata contribute 57.55 CPU-hours, 20.62% of estimated compiler work, despite covering only 5,848 files. This is a source-proxy compiler share; dynamic kernel phase weights require their own measurements. The independent precision panel remains separate.

Lexical density makes the workload difference visible even before timing. After masking comments and strings, the AI corpus contains 1,354,379 kernel-decision sites in

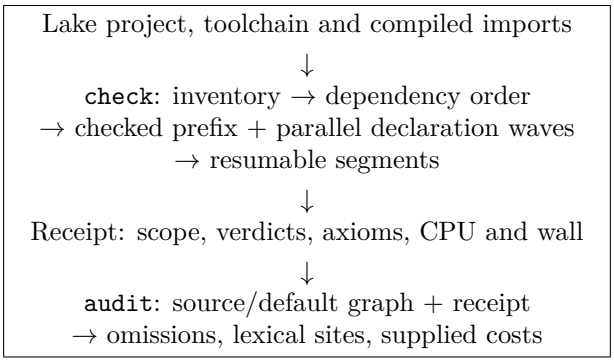


Figure 2: Proof acceptance and its coverage explanation have distinct interfaces. Design implication: a receipt must identify the accepted compiled closure before audit relates it to the source project.

5,848 files; the pinned Mathlib subtree has seven sites in four of 8,529 files. Normalizing by decimal source megabytes gives a 10,668-fold density ratio. An audit should expose these labels alongside measured costs and uncertainty, rather than convert a site count into CPU work.

External cost comparison. The earlier tooling report builds 122,458 modules at `fd4aeeb2` on 32 cloud vCPUs in 13.8 wall-hours [10]. Its approximate 410 CPU-hours comprise roughly 330 hours of compilation and 80 hours of Comparator checking. The compilation CPU total is itself extrapolated from a quiet 26-module laptop sample, with an estimated ±30% uncertainty. Our 279.12 hours therefore compares most directly with its roughly 330-hour compilation estimate. Both 330 and the broader 410-hour total lie inside our wide interval; different commits, module populations and hardware prevent a calibrated agreement claim. The complete cloud run establishes build coverage, whereas our fixed probability sample supplies uniform per-module cost boundaries and an explicit uncertainty estimate at the new pin. The report also measures a 26-module kernel-skipping ablation and profiles roughly 30,900 cloud modules. Those build-side phase measurements complement our target replay-window comparison.

4 ProofServe design

ProofServe turns the cost and coverage findings into separate `check`, `audit` and compatibility-only `serve` interfaces (Figure 2). Its original-kernel route preserves mathematical checking rules while adding explicit coordination and artifact trust.

4.1 Check and resume a dependency closure

A checked prefix amortizes preparation while preserving each module's declared dependency context. `proofserve check <project>` discovers compiled local roots or uses explicit roots, traverses imports, and reports missing artifacts. The checker builds a graph from ordinary declarations' referenced constants. Ready declarations run against a checked frontier using the original kernel operation; the coordinator validates returned identities and commits the next frontier after all workers in a wave join. Nonordinary declarations and their ordinary prerequisites use the original replay path before the remaining graph runs. Unsupported local structure retains conservative fallback.

Module context is a correctness requirement as well as a performance choice. In an earlier prototype, a declaration from a previously visited sibling module satisfied a reference that the incoming module had never imported. The merged context accepted that input while module-local original-kernel replay rejected its unknown constant. The two declared propositions were true; the error was unintended visibility. This finite counterexample motivates explicit dependency context and checked-token identity checks in the integrated tool. A common accepted module-name set is therefore only the first parity check: the comparison must also identify which declaration environment each path actually checked.

Two prototype interventions explain these choices. Correcting our missing persistent marking reduced eight-worker CPU by 14.13% and wall time by 5.13% in a same-executable comparison. Replacing whole-module fallback with a dependency-closed nonordinary prefix reduced a paired wall median from 257.66 to 196.67 seconds (23.67%), with CPU increasing 0.106%. These results identify engineering causes in the prototype; the release ablations test their integrated effect separately. Persistent marking occurs between joined waves. The tool exposes both disabling flags for that evaluation.

Completed segments serialize their successful prefix for reuse. Toolchain, artifacts, Lake lock and checkpoint configuration must match; a directory lock protects a live invocation. An interrupted segment remains incomplete. Receipt v1 distinguishes the accepted prefix, current-invocation CPU and previously completed segment CPU, so a cache hit is identifiable. It records Lean and dependency versions, module-set hash, accepted and rejected modules, reported axiom sets and CPU/wall clocks. A restart experiment charges restoration and restart overhead.

4.2 Audit coverage and explain trust

Audit connects source inventory, the default import graph and the actual checking receipt. `proofserve audit <project>` masks comments and strings before

Closure	Workers	CPU / wall	Set
Mathlib Nat.Basic	Pending	Pending	Pending
Grid	Pending	Pending	Pending
Barrier	Pending	Pending	Pending

Table 1: Release-tool scaling, awaiting the complete committed matrix. Three repeats per worker count support separate CPU and wall summaries. Design implication: assess latency savings together with work inflation; prototype measurements cannot establish release performance.

counting kernel-decision sites and extracts imports. Supported declarative Lake configuration gives default-root coverage; executable or unsupported configuration yields an explicit unknown unless evaluated roots are supplied. Supplied roots retain their supplied status. The receipt contributes its checked compiled closure and reported axioms; a partial receipt reports the last successful prefix. Per-module costs require explicit source-bound observations.

The tool trusts Lean's kernel plus its artifact reader, context metadata, checked-token FFI, quiescent marking and locally generated checkpoints. Internal symbols and serialized metadata require a matching SDK. Unknown toolchains fail clearly or explicitly select the matching official checker, identifying that path and its scope. The current receipt lacks a binding to the current source digest; audit exposes this gap. Thus acceptance, source coverage and axiom dependencies remain separate answers.

5 Evaluation: performance and verification coverage

The fixed evaluation separates established coverage findings from release-tool performance that is still awaiting return. Every admitted system result must name its executable and project commits, toolchain, actual module set and complete timing boundary. Historical prototypes explain causes but cannot populate the release tables.

5.1 RQ3: original-kernel scaling and alternatives

Dependency-aware parallel replay targets elapsed time while preserving an explicit account of total CPU and additional trusted code. The release matrix checks Mathlib Nat.Basic, Grid and Barrier at 1, 2, 4 and 8 workers, with three fresh-checkpoint processes per point. Mathlib here denotes the Nat.Basic closure. At eight workers, Mathlib and Grid each receive three no-persistence and three no-split-fallback runs. Wall/CPU medians and ranges accompany acceptance parity and memory observations (Table 1).

Official default and `--fresh` comparisons require matching ledgers. Lean 4.34.1 already launches module-local tasks; its default path trusts loaded imports and replays local declarations, whereas `--fresh` replays the imported final constant map sequentially [4]. ProofServe's dependency-ordered per-module ledger is a third boundary. The evaluation reports complete command costs and common-scope acceptance; unmatched work is identified before considering speedup ratios.

An independent kernel offers another performance/trust tradeoff. The earlier report completes 416 Comparator configurations in 80.1 active hours, with its longest PlanarPacking check taking about 10.9 hours [10]. A paired sweep finds median nanoda/Lean time 0.21, using four nanoda threads against one Lean thread, excluding export. The pinned nanoda setup, including an enlarged checker stack, accepts 401 of 416 configurations; 15 encounter operational failures such as stack exhaustion or capacity limits, with later revisions recovering most failing cases. Nanoda is a separately implemented kernel [9]; ProofServe instead uses Lean's original kernel plus coordination. The report's result motivates this alternative, without establishing which route is faster under our evaluation conditions.

5.2 RQ4: default targets miss a third of decision sites

The default import graph misses 19.78% of source modules and 33.30% of lexical kernel-decision sites at our corpus pin. It covers 130,834 of 163,097 source modules, leaving 32,263 outside; the omitted files contain 451,008 of 1,354,379 sites. The earlier report finds 30,697 files outside its default graph, about 25% of its 122,459-node source inventory including the root [10]. The revisions and root-count conventions differ, but both demonstrate that a successful default build covers a subset of the source tree. The 33.30% figure describes decision sites, not module count.

Repeated verdicts provide a second, separate kind of evidence. A historical panel checks 2,524 eligible stored-proof targets from three modules at 1, 2, 4 and 8 workers, three repeats each: 30,288 accepted checks and 22,716 one-worker/higher-worker comparisons. Verdicts, diagnostics and allocation counts agree in every observed pair, while elapsed time varies. This supports reproducibility for the tested terms and contexts; release bad-proof CI and interruption parity have their own obligations (Table 2).

Installed-tool static audits already reveal actionable external scope. At pinned revisions, Integer Mult Bounds has 64/64 files in its default graph, Newspeak 105/105, and Solar 12/32. Solar's 20 omitted files are outside the declared Lake import closure; some can instead be inputs to its custom EVM checker. These audits use installed bundle `703055a1`, with project identities and findings in `data/external-projects/`. Inte-

Release acceptance evidence	Current disposition
Three bad-proof classes in CI	CI closure pending
Interrupted vs. uninterrupted	Timed pair pending
Three external projects: native parity	Timing/parity pending
Installed external static audits	3 completed
Package/container clean start	Acceptance pending

Table 2: Functional and external acceptance, with static audit separated from native checking. Design implication: report each completed obligation at its own scope; coverage discovery and proof acceptance are distinct.

ger Mult Bounds discloses AI assistance; this disclosure alone does not identify a wholly AI-generated acceptance workload. Native check/official parity, timing and clean-start results remain pending.

Release validation compares accepted sets and reported axioms at a common scope, requires loose-bound-variable, unknown-constant and wrong-sort inputs to be rejected, and compares a real interrupted/resumed run with an uninterrupted one. External acceptance requires three previously unused public projects, including published AI-generated source. Receipts make this distinction actionable for an automation consumer. A successful process can be paired with its exact accepted module set; a partial run exposes its certified prefix rather than an undifferentiated failure bit. A caller can separately ask whether all intended source modules have compiled evidence and whether a reported axiom is within policy. These questions remain meaningful even when every observed kernel verdict is reproducible. The existing `serve` interface remains Kimina-compatible; service features are outside this evaluation.

Artifact and reproduction. The artifact is at https://github.com/yunwei37/proofserve. The project-facing workflow is `proofserve check /path/to/project`, then `proofserve audit /path/to/project`. The native entry point builds with `cd checker; lake build`; portable audit runs through `tools/audit/audit.py`. The data-only `data/openai-math-4eaee875/summarize.py` recomputes sample summaries. The evidence map records source revisions, plot inputs and the pending release obligations.

6 Limitations and related work

The measurements establish selected cost differences and broad population uncertainty, rather than a universal two-class cost law. Two draws per stratum and heavy tails make the nominal interval fragile; static markers are proxies and lexical density is not runtime frequency. Selected closures overlap, module-local replay trusts imports, and target windows exclude shared preparation. Cross-protocol ratios, cache/order effects, hardware differences and provisional SubexponentialSAT controls limit generalization. Whole-library checking cost and dynamic phase weights remain unmeasured.

Finite verdict agreement does not prove arbitrary-extension safety, future budget sufficiency or soundness of the coordinator. Reported axiom unions retain their actual scope: acceptance can depend on axioms, unsafe/partial exclusions remain visible, and semantic statement faithfulness requires separate review. Current source binding, malicious checkpoint integrity, executable Lake configuration and uncompiled/generated sources require explicit additional evidence. Release scaling, rejection CI, resumption costs and full external acceptance remain incomplete.

Parallel proof checking and asynchronous tasks have precedents in Isabelle, Coq and Lean [13, 1, 3]. Independent checkers and Lake's shipped workflow already address rechecking [6, 7]; the same-corpus tooling report also proposes standardized timing, axiom and hash receipts [10]. Kimina and AXLE supply verification services [12, 14]. Our contribution is the measured boundary between checking and recomputation and its integration into a usable original-kernel tool, rather than new parallelism, receipt or serving mechanisms.

7 Conclusion

Matched measurements show both cheap proof checking and substantial certificate recomputation; a completed probability sample still leaves large uncertainty about total compilation work. Default coverage and repeatable verdicts answer different verification questions. ProofServe makes these boundaries explicit through check, audit and scoped receipts. Its pending release evaluation must establish the practical performance and acceptance of that integration.

References

[1] Bruno Barras, Carst Tankink, and Enrico Tassi. Asynchronous processing of coq documents: from the kernel up to the user interface, 2015.

[2] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction (CADE 28)*, pages 625–635, 2021.

[3] Lean contributors. Lean 4.19.0: Parallel elaboration, 2025.

[4] Lean contributors. Lean 4.34.1 bundled checker: module task dispatch. https://github.com/leanprover/lean4/blob/v4.34.1/src/LeanChecker.lean, 2026.

[5] Lean contributors. Lean 4.34.1 decision-tactic implementation, 2026.

[6] Lean contributors. lean4checker and the built-in leanchecker, 2026.

[7] Lean FRO contributors. Lean 4.35.0 release notes: Lake checking and comparator, 2026.

[8] Lean FRO contributors. The lean fro year 4, part 1 roadmap, 2026.

[9] Nanoda contributors. Nanoda: external lean type checker, 2026. Independent kernel implementation.

[10] nealmcb. Verifying openai/math in lean: cost and checker study. Technical report, revision c9570a82, 2026. Full report inspected 3 November 2026.

[11] OpenAI math contributors. math: Mathematical manuscripts and supporting proof artifacts, 2026. Pinned source release 4eaee87548250b2682affda7d05d947c05554bae; accessed 30 October 2026.

[12] Marco Dos Santos, Hugues de Saxcé, Haiming Wang, Ran Wang, Mantas Bakšys, Mert Ünsal, Junqi Liu, Zhengying Liu, and Jia Li. Kimina lean server: A high-performance lean server for large-scale verification. In *Fifth Workshop on Mathematical Reasoning and AI (MATH-AI) at NeurIPS*, 2025.

[13] Makarius Wenzel. Parallel proof checking in isabelle/isar. In *ACM SIGSAM Workshop on Programming Languages for Mechanized Mathematics Systems (PLMMS)*, 2009.

[14] Jimmy Xin, Alex Schneidman, Chris Cummins, Karun Ram, Srihari Ganesh, and Jannis Limperg. AXLE: A cloud infrastructure for lean 4 theorem proving utilities, 2026. Version 1; accepted at the 3rd AI for Math Workshop, ICML 2026.